

Git Lifecycle: Branching, Pushing & Automation

Here is the exact, step-by-step walkthrough you can use to demonstrate the entire Git lifecycle to your class. Workflow requires strict chronological execution. Missing or reordering a step will cause terminal errors for the students.

Step 1: Initial Setup

Configure global identity

Before doing anything, Git needs to know who is making the changes. Have students run this once on their machines.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

To verify it worked:

```
git config --list
```

Step 2: Initialize the Repository

Create the working directory

Have students create a new folder for their C++ project and initialize it as a Git repository.

```
mkdir my-cpp-project
cd my-cpp-project
git init
```

Note for the class: This creates a hidden .git folder. This is where all the time-travel magic is permanently stored.

Step 3: The First Commit

Staging and committing

Create a basic C++ file, move it to the staging area, and commit it.

- Create a file named `main.cpp`:

```
#include <iostream>
int main() {
    std::cout << "Hello Version Control!" << std::endl;
    return 0;
}
```

- Check the status (it will show in red as untracked):

```
git status
```

- Add it to the staging area:

```
git add main.cpp
```

- Save it permanently to the timeline:

```
git commit -m "Initial commit: Add basic main function"
```

Step 4: Setting up `.gitignore`

Ignoring build artifacts

When you compile C++ code (e.g., `g++ main.cpp -o app`), it creates a binary file (`app.exe` or `app`). We never commit binaries to Git.

```
# Ignore compiled binaries
*.exe
*.out
*.o
.DS_Store
```

- Stage and commit the ignore file:

```
git add .gitignore
git commit -m "Add .gitignore for C++ builds"
```

Step 5: Create a Feature Branch

Safe experimentation

Let's safely add a new feature without touching the working main code.

```
git switch -c feature-math
```

- Modify main.cpp to include some math logic.
- Stage and commit the new feature:

```
git add main.cpp  
git commit -m "Add math functionality"
```

Step 6: Merging Branches

Combining parallel timelines

The feature works. Now, bring it back into the main project.

```
git switch main  
git merge feature-math  
git branch -d feature-math
```

Step 7: Resolving a Merge Conflict

Handling competing changes

Show them what happens when things go wrong.

- On main, change line 3 of main.cpp to say "Hello World", then commit it.
- Create a branch called conflict-test, change that exact same line to "Hello Class", and commit it.
- Switch to main and try to merge conflict-test. Git will throw a Merge Conflict.

```
<<<<<<< HEAD  
    std::cout << "Hello World!" << std::endl;  
=====
```

```
std::cout << "Hello Class!" << std::endl;  
>>>>>> conflict-test
```

- The Fix: Manually delete the markers, keep the line you want, save the file, and run:

```
git add main.cpp  
git commit -m "Resolve merge conflict in main.cpp"
```

Step 8: Connecting to GitHub

Pushing to the cloud

Finally, back up the local project to the internet.

```
git remote add origin https://github.com/USERNAME/my-cpp-project.git  
git push -u origin main
```

Step 9: Advanced Branching & Stashing

Handling temporary workflows

Sometimes you need to switch branches quickly but aren't ready to commit.

```
git stash  
git switch other-branch  
# ...do work, then come back...  
git switch feature-math  
git stash pop
```

Step 10: Fetching and Pulling

Syncing with remote repositories

To safely check for updates without overwriting your files, use `fetch`. To directly merge them, use `pull`.

```
git fetch
git pull origin main
```

Step 11: Remote Management

Managing URLs and connections

If you need to change the repository URL or view existing connections:

```
git remote -v
git remote set-url origin NEW_URL
```

Step 12: Automating the Push Workflow

Using batch scripts (.bat)

You can create a quick Windows .bat file to automate adding, committing, and pushing code.

```
@echo off
echo Building...
git add .
git commit -m "Auto commit"
git push origin main
pause
```